Day13 系统测试与调试

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 13 天（8 课时）
> 主题：系统测试与调试，确保代码质量和稳定性

一、本日目标

1. 理解软件测试的意义和分类；
2. 掌握单元测试（unittest/pytest）；
3. 掌握集成测试方法；
4. 掌握调试技巧（pdb/print/logging）；
5. 掌握异常处理；
6. 掌握代码规范与静态检查；
7. 掌握版本控制（git）；
8. 完成系统测试实战。

二、知识要点

- 软件测试：单元测试、集成测试、系统测试、验收测试；
- 单元测试：unittest（标准库）、pytest（第三方）；
- 调试：print 调试、logging 日志、pdb 调试器；
- 异常处理：try-except-else-finally、自定义异常；
- 代码规范：PEP8、flake8、pylint；
- 版本控制：git 基本操作、分支管理、提交规范。

课时1：软件测试基础

1.1 课时目标

理解软件测试的意义、分类和测试流程。

1.2 知识点深度讲解

1.2.1 为什么需要软件测试

软件测试的意义：

1. **发现缺陷**：在交付前发现代码中的bug；
2. **保证质量**：确保系统功能正确、性能达标；
3. **降低风险**：减少线上故障和用户投诉；
4. **便于重构**：有测试保护，重构代码更安全；
5. **文档作用**：测试用例是最好的使用文档。

1.2.2 软件测试分类

1. **按测试阶段**：
   - 单元测试：测试单个函数/类；
   - 集成测试：测试模块之间的交互；
   - 系统测试：测试整个系统；
   - 验收测试：用户确认系统满足需求。
2. **按是否执行代码**：
   - 静态测试：不执行代码，检查代码规范（flake8/pylint）；
   - 动态测试：执行代码，检查运行结果。
3. **按测试方法**：
   - 白盒测试：了解内部结构，测试代码路径；
   - 黑盒测试：不了解内部结构，测试输入输出；
   - 灰盒测试：介于白盒和黑盒之间。

1.2.3 测试流程

1. **测试计划**：确定测试范围、策略、资源；
2. **测试设计**：编写测试用例；
3. **测试执行**：运行测试用例，记录结果；
4. **缺陷管理**：记录、跟踪、修复缺陷；
5. **测试报告**：总结测试结果和质量评估。

1.2.4 测试用例设计

好的测试用例应包含：

1. **正常情况**：输入合法数据，验证正确输出；
2. **边界情况**：输入边界值（如0、最大值、空值）；
3. **异常情况**：输入非法数据，验证错误处理；
4. **性能情况**：大数据量下的性能表现。

1.3 完整代码（测试用例演示）

```
# ============================================================
# 课时1：软件测试基础 - 测试用例演示
# ============================================================

import numpy as np


# 被测函数：Haversine距离
def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """计算球面上两点之间的距离。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
    c = 2 * np.arcsin(np.sqrt(a))
    d = R * c
    return d * 1000 if unit == "m" else d


def test_haversine_normal():
    """正常情况：两点距离。"""
    # 沈阳到北京（约680km）
    d = haversine(123.43, 41.80, 116.40, 39.90)
    print(f"  沈阳到北京: {d:.1f} km (预期约680km)")
    assert 650 < d < 720, f"距离异常: {d}"


def test_haversine_same_point():
    """边界情况：同一点距离为0。"""
    d = haversine(123.43, 41.80, 123.43, 41.80)
    print(f"  同一点距离: {d:.6f} km (预期0)")
    assert d == 0, f"同一点距离应为0: {d}"


def test_haversine_unit_m():
```

```
    """正常情况：单位为米。"""
    d_km = haversine(123.43, 41.80, 123.44, 41.81)
    d_m = haversine(123.43, 41.80, 123.44, 41.81, unit="m")
    print(f"  距离: {d_km:.4f} km = {d_m:.1f} m")
    assert abs(d_km * 1000 - d_m) < 0.01, "单位转换错误"


def test_haversine_vectorized():
    """正常情况：向量化计算。"""
    lons = np.array([123.44, 123.45, 123.46])
    lats = np.array([41.81, 41.82, 41.83])
    d = haversine(123.43, 41.80, lons, lats)
    print(f"  向量化距离: {d}")
    assert len(d) == 3, "向量化结果长度错误"


def test_haversine_antipodal():
    """边界情况：对跖点（地球直径）。"""
    d = haversine(0, 0, 180, 0)
    print(f"  对跖点距离: {d:.1f} km (地球直径约20015km)")
    assert 19000 < d < 21000, f"对跖点距离异常: {d}"


def run_all_tests():
    """运行所有测试。"""
    print("=" * 60)
    print("Haversine距离函数测试")
    print("=" * 60)

    tests = [
        ("正常情况-两点距离", test_haversine_normal),
        ("边界情况-同一点", test_haversine_same_point),
        ("正常情况-单位米", test_haversine_unit_m),
        ("正常情况-向量化", test_haversine_vectorized),
        ("边界情况-对跖点", test_haversine_antipodal),
    ]

    passed = 0
```

```
    failed = 0
    for name, test_func in tests:
        print(f"\n[{name}]")
        try:
            test_func()
            print(f"  ✓ 通过")
            passed += 1
        except AssertionError as e:
            print(f"  ✗ 失败: {e}")
            failed += 1
        except Exception as e:
            print(f"  ✗ 错误: {type(e).__name__}: {e}")
            failed += 1

    print(f"\n{'='*60}")
    print(f"测试结果: 通过{passed}个, 失败{failed}个, 共{passed+failed}个")
    print(f"{'='*60}")


if __name__ == "__main__":
    run_all_tests()
```

1.4 运行结果与解读

运行后展示5个测试用例的执行结果：
1. 正常情况-两点距离：沈阳到北京约680km；
2. 边界情况-同一点：距离为0；
3. 正常情况-单位米：km和m转换正确；
4. 正常情况-向量化：返回3个距离；
5. 边界情况-对跖点：地球直径约20015km。

> 关键理解：
> 1. 好的测试用例覆盖正常、边界、异常情况；
> 2. 断言（assert）验证结果是否符合预期；
> 3. 测试用例是最好的文档，展示函数的使用方法；
> 4. 自动化测试可以反复运行，确保修改不引入新bug。

1.5 当堂练习

1. 为 `haversine` 函数增加一个异常测试（输入非法坐标）；
2. 思考：为什么边界情况测试很重要？（提示：bug常出现在边界）
3. 尝试为 `kmeans` 函数设计测试用例。

1.6 课时小结

- 软件测试意义：发现缺陷、保证质量、降低风险、便于重构；
- 测试分类：单元/集成/系统/验收，静态/动态，白盒/黑盒；
- 测试流程：计划→设计→执行→缺陷管理→报告；
- 测试用例：正常情况、边界情况、异常情况、性能情况；
- 断言验证：assert 验证结果是否符合预期。

课时2：单元测试（unittest/pytest）

2.1 课时目标

掌握 unittest 和 pytest 单元测试框架。

2.2 知识点深度讲解

2.2.1 unittest（标准库）

unittest 是 Python 标准库的测试框架，基于JUnit设计：
1. **测试类**：继承 `unittest.TestCase`；
2. **测试方法**：以 `test_` 开头；
3. **断言方法**：`assertEqual`、`assertTrue`、`assertRaises` 等；
4. **setUp/tearDown**：测试前后的准备和清理；
5. **运行**：`unittest.main()` 或 `python -m unittest`。

2.2.2 pytest（第三方）

pytest 是更简洁的测试框架：
1. **无需继承**：普通函数即可，以 `test_` 开头；
2. **assert 直接用**：不需要 `assertEqual`，直接用 `assert`；
3. **fixture**：`@pytest.fixture` 提供测试前置数据；
4. **参数化**：`@pytest.mark.parametrize` 批量测试；
5. **插件丰富**：pytest-cov（覆盖率）、pytest-xdist（并行）等。

2.2.3 断言方法对比

unittest	pytest	说明
assertEqual(a, b)	assert a == b	相等
assertTrue(x)	assert x	为真
assertIn(a, b)	assert a in b	包含
assertRaises(Exc)	pytest.raises(Exc)	抛出异常
assertAlmostEqual(a, b)	abs(a-b) < eps	近似相等

2.3 完整代码（unittest 示例）

```
# ============================================================
# 课时2：单元测试 - unittest 示例
# 文件：test_utils.py
# 运行：python -m unittest test_utils.py -v
# ============================================================

import unittest
import numpy as np
import sys
import os

# 添加代码目录
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "project_code"))
from utils import haversine


class TestHaversine(unittest.TestCase):
    """Haversine距离函数测试类。"""

    def setUp(self):
        """每个测试方法执行前的准备。"""
        self.shenyang = (123.43, 41.80)  # 沈阳
        self.beijing = (116.40, 39.90)    # 北京

    def tearDown(self):
        """每个测试方法执行后的清理。"""
        pass

    def test_normal_distance(self):
        """正常情况：沈阳到北京距离。"""
        d = haversine(*self.shenyang, *self.beijing)
        self.assertGreater(d, 650)
        self.assertLess(d, 720)

    def test_same_point(self):
        """边界情况：同一点距离为0。"""
```

```
        d = haversine(*self.shenyang, *self.shenyang)
        self.assertEqual(d, 0.0)

    def test_unit_meter(self):
        """正常情况：单位为米。"""
        d_km = haversine(*self.shenyang, 123.44, 41.81)
        d_m = haversine(*self.shenyang, 123.44, 41.81, unit="m")
        self.assertAlmostEqual(d_km * 1000, d_m, places=2)

    def test_vectorized(self):
        """正常情况：向量化计算。"""
        lons = np.array([123.44, 123.45, 123.46])
        lats = np.array([41.81, 41.82, 41.83])
        d = haversine(*self.shenyang, lons, lats)
        self.assertEqual(len(d), 3)
        self.assertTrue(all(d > 0))

    def test_antipodal(self):
        """边界情况：对跖点。"""
        d = haversine(0, 0, 180, 0)
        self.assertGreater(d, 19000)
        self.assertLess(d, 21000)


class TestDataClean(unittest.TestCase):
    """数据清洗函数测试类。"""

    def test_remove_empty(self):
        """测试去空值。"""
        import pandas as pd
        df = pd.DataFrame({
            "imsi": ["111", "222", "333"],
            "lac_id": ["100", "", "300"],
            "cell_id": ["1000", "2000", ""],
        })
        # 去空
        df_clean = df[(df["lac_id"] != "") & (df["cell_id"] != "")]
        self.assertEqual(len(df_clean), 1)
```

```
        self.assertEqual(df_clean.iloc[0]["imsi"], "111")

    def test_filter_by_date(self):
        """测试按日期过滤。"""
        import pandas as pd
        df = pd.DataFrame({
            "time": ["20181003120000", "20181004120000", "20181003130000"],
        })
        target_date = "20181003"
        df_filtered = df[df["time"].str.startswith(target_date)]
        self.assertEqual(len(df_filtered), 2)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

2.4 完整代码（pytest 示例）

```
# ============================================================
# 课时2：单元测试 - pytest 示例
# 文件：test_utils_pytest.py
# 运行：pytest test_utils_pytest.py -v
# 安装：pip install pytest
# ============================================================

import pytest
import numpy as np
import sys
import os

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "project_code"))
from utils import haversine


# ============================================================
# fixture：测试前置数据
# ============================================================
@pytest.fixture
def cities():
    """提供城市坐标。"""
    return {
        "shenyang": (123.43, 41.80),
        "beijing": (116.40, 39.90),
        "shanghai": (121.47, 31.23),
    }


# ============================================================
# 普通测试函数
# ============================================================
def test_normal_distance(cities):
    """正常情况：沈阳到北京距离。"""
    d = haversine(*cities["shenyang"], *cities["beijing"])
    assert 650 < d < 720
```

```
def test_same_point(cities):
    """边界情况：同一点距离为0。"""
    d = haversine(*cities["shenyang"], *cities["shenyang"])
    assert d == 0.0


def test_unit_meter(cities):
    """正常情况：单位为米。"""
    d_km = haversine(*cities["shenyang"], 123.44, 41.81)
    d_m = haversine(*cities["shenyang"], 123.44, 41.81, unit="m")
    assert abs(d_km * 1000 - d_m) < 0.01


def test_vectorized(cities):
    """正常情况：向量化计算。"""
    lons = np.array([123.44, 123.45, 123.46])
    lats = np.array([41.81, 41.82, 41.83])
    d = haversine(*cities["shenyang"], lons, lats)
    assert len(d) == 3
    assert all(d > 0)


# ============================================================
# 参数化测试：批量测试多个输入
# ============================================================
@pytest.mark.parametrize("lon1,lat1,lon2,lat2,expected_min,expected_max", [
    (123.43, 41.80, 116.40, 39.90, 650, 720),   # 沈阳-北京
    (123.43, 41.80, 121.47, 31.23, 1000, 1300), # 沈阳-上海
    (0, 0, 0, 0, -0.1, 0.1),                      # 同一点
    (0, 0, 180, 0, 19000, 21000),                  # 对跖点
])
def test_parametrized(lon1, lat1, lon2, lat2, expected_min, expected_max):
    """参数化测试：多组输入。"""
    d = haversine(lon1, lat1, lon2, lat2)
    assert expected_min < d < expected_max
```

```
# ============================================================
# 异常测试
# ============================================================
def test_invalid_unit():
    """异常情况：非法单位。"""
    # 当前函数不校验单位，测试预期行为
    d = haversine(0, 0, 1, 1, unit="invalid")
    # 非法单位会走else分支，返回km
    assert d > 0



# ============================================================
# 运行：pytest test_utils_pytest.py -v
# ============================================================
```

2.5 运行结果与解读

- unittest 运行：`python -m unittest test_utils.py -v`，显示每个测试方法的结果；
- pytest 运行：`pytest test_utils_pytest.py -v`，显示更详细的结果，包括参数化测试的每组输入。

> 关键理解：
> 1. unittest 需要继承 TestCase，pytest 普通函数即可；
> 2. pytest 用普通 assert，更简洁；
> 3. fixture 提供测试前置数据，避免重复；
> 4. 参数化测试可以批量测试多组输入；
> 5. 推荐用 pytest，更简洁更强大。

2.6 当堂练习

1. 用 pytest 为 `kmeans` 函数编写测试用例；
2. 用参数化测试测试多组K值；
3. 思考：fixture 和 setUp 有什么区别？（提示：fixture 更灵活，可以参数化）

2.7 课时小结

- unittest：标准库，继承 TestCase，assertEqual 等断言；

- pytest：第三方，普通函数，普通 assert，更简洁；
- fixture：测试前置数据，避免重复；
- 参数化：@pytest.mark.parametrize 批量测试；
- 推荐用 pytest，更简洁更强大。

课时3：集成测试

3.1 课时目标

掌握集成测试方法，测试模块之间的交互。

3.2 知识点深度讲解

3.2.1 集成测试的意义

集成测试的意义：
1. **验证模块交互**：单元测试只测单个模块，集成测试测模块之间的交互；
2. **发现接口问题**：模块之间的数据传递、格式转换是否正确；
3. **验证数据流**：数据从输入到输出的完整流程是否正确；
4. **发现集成bug**：单个模块正常，但组合后出现的bug。

3.2.2 集成测试策略

1. **自顶向下**：从主模块开始，逐步集成子模块；
2. **自底向上**：从底层模块开始，逐步集成上层模块；
3. **大爆炸**：所有模块一次集成（不推荐，难以定位问题）；
4. **持续集成**：每次代码提交都自动运行集成测试。

3.2.3 本系统的集成测试

本系统的集成测试重点：
1. **数据清洗→阶段1**：cleaned_data.csv 是否能被 stage1 正确读取；
2. **阶段1→阶段2**：stage1_travel_points.csv 是否能被 stage2 正确读取；
3. **阶段2→阶段3**：stage2_cluster.csv 是否能被 stage3 正确读取；
4. **阶段3→目的/方式**：stage3_label.csv 是否能被 purpose/mode 正确读取；
5. **全流程**：main.py 一键运行是否成功。

3.3 完整代码（集成测试示例）

```
# ============================================================
# 课时3：集成测试示例
# 文件：test_integration.py
# 运行：pytest test_integration.py -v
# ============================================================

import pytest
import os
import sys
import pandas as pd
import numpy as np

# 添加代码目录
CODE_DIR = os.path.join(os.path.dirname(__file__), "project_code")
sys.path.insert(0, CODE_DIR)

from config import (
    CLEANED_DATA_FILE, STAGE1_TRAVEL_POINTS_FILE,
    STAGE2_CLUSTER_FILE, STAGE3_LABEL_FILE,
    PURPOSE_RESULT_FILE, MODE_RESULT_FILE
)



# ============================================================
# fixture：检查文件存在
# ============================================================
@pytest.fixture(scope="module")
def check_files():
    """检查所有结果文件是否存在。"""
    files = {
        "cleaned": CLEANED_DATA_FILE,
        "stage1": STAGE1_TRAVEL_POINTS_FILE,
        "stage2": STAGE2_CLUSTER_FILE,
        "stage3": STAGE3_LABEL_FILE,
        "purpose": PURPOSE_RESULT_FILE,
        "mode": MODE_RESULT_FILE,
```

```
    }
    return files


# ============================================================
# 测试1：数据清洗输出
# ============================================================
class TestDataCleanOutput:
    """数据清洗输出测试。"""

    def test_file_exists(self, check_files):
        """测试文件存在。"""
        assert os.path.exists(check_files["cleaned"]), f"文件不存在:
{check_files['cleaned']}"

    def test_file_not_empty(self, check_files):
        """测试文件非空。"""
        df = pd.read_csv(check_files["cleaned"], nrows=5)
        assert len(df) > 0, "文件为空"

    def test_columns_exist(self, check_files):
        """测试必要列存在。"""
        df = pd.read_csv(check_files["cleaned"], nrows=5)
        required_cols = ["imsi", "time", "lac_id", "cell_id", "lon", "lat"]
        for col in required_cols:
            assert col in df.columns, f"缺少列: {col}"

    def test_no_null_coords(self, check_files):
        """测试经纬度无空值。"""
        df = pd.read_csv(check_files["cleaned"])
        assert df["lon"].notna().all(), "存在空经度"
        assert df["lat"].notna().all(), "存在空纬度"

    def test_date_correct(self, check_files):
        """测试日期正确。"""
        df = pd.read_csv(check_files["cleaned"])
        assert df["time"].str.startswith("20181003").all(), "存在非目标日期数据"
```

```
# ============================================================
# 测试2：阶段1输出
# ============================================================
class TestStage1Output:
    """阶段1输出测试。"""

    def test_file_exists(self, check_files):
        assert os.path.exists(check_files["stage1"])

    def test_has_speed_column(self, check_files):
        df = pd.read_csv(check_files["stage1"], nrows=5)
        assert "speed" in df.columns, "缺少speed列"

    def test_travel_points_speed(self, check_files):
        """测试出行点速度大于阈值。"""
        df = pd.read_csv(check_files["stage1"])
        assert (df["speed"] > 8).all(), "存在速度<=8的出行点"


# ============================================================
# 测试3：阶段2输出
# ============================================================
class TestStage2Output:
    """阶段2输出测试。"""

    def test_file_exists(self, check_files):
        assert os.path.exists(check_files["stage2"])

    def test_has_cluster_column(self, check_files):
        df = pd.read_csv(check_files["stage2"], nrows=5)
        assert "cluster" in df.columns, "缺少cluster列"

    def test_cluster_labels_valid(self, check_files):
        """测试聚类标签有效。"""
        df = pd.read_csv(check_files["stage2"])
        assert df["cluster"].min() >= 0, "存在负聚类标签"
```

```
# ============================================================
# 测试4：阶段3输出
# ============================================================
class TestStage3Output:
    """阶段3输出测试。"""

    def test_file_exists(self, check_files):
        assert os.path.exists(check_files["stage3"])

    def test_has_label_column(self, check_files):
        df = pd.read_csv(check_files["stage3"], nrows=5)
        assert "label" in df.columns, "缺少label列"

    def test_label_values_valid(self, check_files):
        """测试标签值有效（0或1）。"""
        df = pd.read_csv(check_files["stage3"])
        assert set(df["label"].unique()).issubset({0, 1}), "存在非法标签值"

    def test_both_labels_exist(self, check_files):
        """测试驻留和出行标签都存在。"""
        df = pd.read_csv(check_files["stage3"])
        assert 0 in df["label"].values, "没有驻留标签"
        assert 1 in df["label"].values, "没有出行标签"


# ============================================================
# 测试5：数据流转测试（阶段间数据一致性）
# ============================================================
class TestDataFlow:
    """数据流转测试。"""

    def test_stage1_users_subset_of_cleaned(self, check_files):
        """测试阶段1用户是清洗后用户的子集。"""
        df_clean = pd.read_csv(check_files["cleaned"], usecols=["imsi"])
        df_stage1 = pd.read_csv(check_files["stage1"], usecols=["imsi"])
        clean_users = set(df_clean["imsi"].unique())
        stage1_users = set(df_stage1["imsi"].unique())
```

```
        assert stage1_users.issubset(clean_users), "阶段1存在不在清洗结果中的用户"

    def test_stage3_rows_match_cleaned(self, check_files):
        """测试阶段3行数与清洗结果一致。"""
        df_clean = pd.read_csv(check_files["cleaned"])
        df_stage3 = pd.read_csv(check_files["stage3"])
        assert len(df_clean) == len(df_stage3), \
            f"行数不一致: cleaned={len(df_clean)}, stage3={len(df_stage3)}"


# ============================================================
# 运行: pytest test_integration.py -v
# ============================================================
```

3.4 运行结果与解读

运行后展示各模块输出测试和数据流转测试的结果：
1. 数据清洗输出：文件存在、非空、必要列、无空坐标、日期正确；
2. 阶段1输出：文件存在、有speed列、出行点速度>8；
3. 阶段2输出：文件存在、有cluster列、标签有效；
4. 阶段3输出：文件存在、有label列、标签值0/1、两种标签都存在；
5. 数据流转：阶段1用户是清洗结果的子集、阶段3行数与清洗结果一致。

> 关键理解：
> 1. 集成测试验证模块之间的交互和数据流转；
> 2. 测试文件存在、格式正确、数据一致性；
> 3. 数据流转测试验证上游输出能被下游正确读取；
> 4. 集成测试是持续集成的重要组成部分。

3.5 当堂练习

1. 为 purpose.py 输出编写集成测试；
2. 编写一个全流程测试（运行 main.py 后检查所有输出）；
3. 思考：集成测试和单元测试的区别？（提示：单元测试测单个函数，集成测试测模块交互）

3.6 课时小结

- 集成测试：验证模块之间的交互和数据流转；

- 测试策略：自顶向下、自底向上、持续集成；
- 测试内容：文件存在、格式正确、数据一致性、数据流转；
- 数据流转测试：验证上游输出能被下游正确读取；
- 集成测试是持续集成的重要组成部分。

课时4：调试技巧（pdb/print/logging）

4.1 课时目标

掌握 Python 调试技巧，快速定位和修复bug。

4.2 知识点深度讲解

4.2.1 调试的基本思路

调试的基本思路：
1. **复现问题**：稳定复现bug，才能调试；
2. **定位问题**：缩小范围，找到出错的代码行；
3. **分析原因**：理解为什么出错；
4. **修复问题**：修改代码，修复bug；
5. **验证修复**：确认bug已修复，没有引入新问题。

4.2.2 print 调试（最简单）

print 调试是最常用的调试方法：

```
print(f"变量值：{var}")
print(f"函数入口，参数：{args}")
print(f"函数出口，返回：{result}")
```

优点：简单直接，不需要额外工具；
缺点：调试完成后需要删除，容易遗漏。

4.2.3 logging 日志（推荐）

logging 是 Python 标准库的日志模块：
1. **日志级别**：DEBUG、INFO、WARNING、ERROR、CRITICAL；
2. **灵活配置**：可以输出到控制台、文件、邮件等；
3. **不删除**：调试完成后保留日志，改级别即可；
4. **格式化**：可以包含时间、级别、文件名、行号等。

```
import logging
logging.basicConfig(level=logging.DEBUG,
                    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)
logger.debug("调试信息")
logger.info("普通信息")
logger.warning("警告信息")
logger.error("错误信息")
```

4.2.4 pdb 调试器（强大）

pdb 是 Python 标准库的调试器，可以：

1. **设置断点**：`breakpoint()` 或 `pdb.set_trace()`；
2. **单步执行**：`n`（next）、`s`（step）；
3. **查看变量**：`p var` 或直接输入变量名；
4. **继续执行**：`c`（continue）；
5. **查看代码**：`l`（list）；
6. **调用栈**：`w`（where）。

常用命令：

命令	说明
n (next)	执行下一行（不进入函数）
s (step)	执行下一行（进入函数）
c (continue)	继续执行到下一个断点
p var	打印变量值
l (list)	查看当前代码
w (where)	查看调用栈
u (up)	上移调用栈
d (down)	下移调用栈
q (quit)	退出调试器

4.3 完整代码（调试技巧演示）

```
# ============================================================
# 课时4：调试技巧演示
# ============================================================

import logging
import pdb
import numpy as np
import pandas as pd



# ============================================================
# 1. logging 日志配置
# ============================================================
def demo_logging():
    """演示 logging 日志。"""
    print("=" * 60)
    print("1. logging 日志演示")
    print("=" * 60)

    # 配置日志
    logging.basicConfig(
        level=logging.DEBUG,
        format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
        datefmt='%Y-%m-%d %H:%M:%S'
    )
    logger = logging.getLogger("demo")

    # 不同级别的日志
    logger.debug("这是调试信息（变量值、函数入口等）")
    logger.info("这是普通信息（程序进度）")
    logger.warning("这是警告信息（可能有问题）")
    logger.error("这是错误信息（出错了）")
    logger.critical("这是严重错误（程序无法继续）")

    # 实际应用：在函数中记录日志
    def calculate_speed(dist, time_diff):
```

```
        """计算速度。"""
        logger.debug(f"calculate_speed 入口: dist={dist}, time_diff={time_diff}")

        if time_diff == 0:
            logger.warning("时间差为0，返回NaN")
            return np.nan

        speed = dist * 3600 / time_diff
        logger.debug(f"calculate_speed 出口: speed={speed}")

        if speed > 200:
            logger.warning(f"速度异常: {speed:.1f} km/h")

        return speed

    # 测试
    print("\n  测试 calculate_speed:")
    print(f"    正常: {calculate_speed(1.0, 300):.1f} km/h")
    print(f"    时间差为0: {calculate_speed(1.0, 0)}")
    print(f"    速度异常: {calculate_speed(100.0, 100):.1f} km/h")


# ============================================================
# 2. print 调试演示
# ============================================================
def demo_print_debug():
    """演示 print 调试。"""
    print("\n" + "=" * 60)
    print("2. print 调试演示")
    print("=" * 60)

    def buggy_function(data):
        """有bug的函数。"""
        result = []
        for i, x in enumerate(data):
            # print 调试：查看每次循环的变量
            # print(f"i={i}, x={x}, type(x)={type(x)}")
            result.append(x * 2)
```

```
        return result

    # 正常数据
    data1 = [1, 2, 3]
    print(f"  正常数据: {buggy_function(data1)}")

    # 异常数据（包含字符串）
    data2 = [1, "2", 3]
    try:
        print(f"  异常数据: {buggy_function(data2)}")
    except TypeError as e:
        print(f"  异常数据报错: {e}")
        print("  调试思路：在循环中 print i, x, type(x)，定位到 x='2' 时出错")


# ============================================================
# 3. pdb 调试演示
# ============================================================
def demo_pdb():
    """演示 pdb 调试（需要手动运行，不能在脚本中自动演示）。"""
    print("\n" + "=" * 60)
    print("3. pdb 调试演示")
    print("=" * 60)

    print("""
  pdb 调试使用方法：

  1. 在代码中设置断点：
     import pdb
     pdb.set_trace()  # 或 breakpoint() (Python 3.7+)

  2. 运行程序，程序会在断点处暂停

  3. 使用命令调试：
     n (next)     - 执行下一行（不进入函数）
     s (step)     - 执行下一行（进入函数）
     c (continue) - 继续执行到下一个断点
     p var        - 打印变量值
```

```
    l (list)     - 查看当前代码
    w (where)    - 查看调用栈
    q (quit)     - 退出调试器

  4. 示例：
    def buggy_func():
        x = 1
        y = 0
        breakpoint()  # 程序暂停在这里
        z = x / y     # 可以用 p x, p y 查看变量
        return z
    """)

    # 实际示例（不真正进入pdb，只展示代码结构）
    def divide(a, b):
        """除法函数。"""
        # breakpoint()  # 取消注释即可进入调试
        if b == 0:
            return float('inf')
        return a / b

    print(f"  divide(10, 2) = {divide(10, 2)}")
    print(f"  divide(10, 0) = {divide(10, 0)}")


# ============================================================
# 4. 常见bug类型和调试方法
# ============================================================
def demo_common_bugs():
    """演示常见bug类型。"""
    print("\n" + "=" * 60)
    print("4. 常见bug类型和调试方法")
    print("=" * 60)

    # Bug 1: 类型错误
    print("\n  Bug 1: 类型错误（TypeError）")
    print("  原因：字符串和数字相加")
    print("  调试：print(type(var)) 查看类型")
```

```
print("  修复：int(var) 或 str(var) 转换")

# Bug 2: 索引越界
print("\n  Bug 2: 索引越界（IndexError）")
print("  原因：访问不存在的索引")
data = [1, 2, 3]
try:
    print(data[5])
except IndexError as e:
    print(f"  报错：{e}")
    print("  调试：print(len(data)) 查看长度")
    print("  修复：检查索引范围")

# Bug 3: 键不存在
print("\n  Bug 3: 键不存在（KeyError）")
print("  原因：访问字典中不存在的键")
d = {"a": 1}
try:
    print(d["b"])
except KeyError as e:
    print(f"  报错：{e}")
    print("  调试：print(d.keys()) 查看所有键")
    print("  修复：d.get('b', default) 或 'b' in d")

# Bug 4: 除零错误
print("\n  Bug 4: 除零错误（ZeroDivisionError）")
print("  原因：除数为0")
try:
    print(1 / 0)
except ZeroDivisionError as e:
    print(f"  报错：{e}")
    print("  调试：print(denominator) 查看除数")
    print("  修复：if denominator == 0: 处理")

# Bug 5: pandas SettingWithCopyWarning
print("\n  Bug 5: pandas SettingWithCopyWarning")
print("  原因：在切片上修改数据")
print("  调试：检查是否用了 df[df['x']>0]['y'] = 1")
```

```
    print("  修复：用 df.loc[df['x']>0, 'y'] = 1")


if __name__ == "__main__":
    print("=" * 70)
    print("  调试技巧演示")
    print("=" * 70)

    demo_logging()
    demo_print_debug()
    demo_pdb()
    demo_common_bugs()

    print("\n" + "=" * 70)
    print("  调试技巧演示完成！")
    print("=" * 70)
```

4.4 运行结果与解读

运行后展示4种调试技巧：
1. logging 日志：5种级别，实际应用在函数中记录入口/出口/异常；
2. print 调试：最简单的调试方法，定位类型错误；
3. pdb 调试器：强大的交互式调试，常用命令说明；
4. 常见bug类型：TypeError、IndexError、KeyError、ZeroDivisionError、SettingWithCopyWarning。

> 关键理解：
> 1. 调试思路：复现→定位→分析→修复→验证；
> 2. print 调试最简单，但调试完成后需要删除；
> 3. logging 更专业，保留日志，改级别即可；
> 4. pdb 最强大，可以单步执行、查看变量、查看调用栈；
> 5. 常见bug都有典型的报错信息和修复方法。

4.5 当堂练习

1. 用 logging 改造 data_clean.py，在每步前后记录日志；
2. 故意在代码中引入一个bug，用 pdb 调试并修复；
3. 思考：什么时候用 print，什么时候用 logging，什么时候用 pdb？

4.6 课时小结

- 调试思路：复现→定位→分析→修复→验证；
- print 调试：最简单，适合快速定位；
- logging 日志：专业，5种级别，保留日志；
- pdb 调试器：强大，单步执行、查看变量、调用栈；
- 常见bug：TypeError、IndexError、KeyError、ZeroDivisionError、SettingWithCopyWarning。

课时5：异常处理

5.1 课时目标

掌握 Python 异常处理，编写健壮的代码。

5.2 知识点深度讲解

5.2.1 异常处理的意义

异常处理的意义：
1. **程序健壮**：遇到错误不会崩溃，能优雅处理；
2. **用户友好**：给出清晰的错误提示，而不是堆栈跟踪；
3. **资源释放**：确保文件、连接等资源正确释放；
4. **便于调试**：记录错误信息，便于定位问题；
5. **业务逻辑**：根据错误类型执行不同的处理逻辑。

5.2.2 try-except-else-finally

```
try:
    # 可能出错的代码
    result = risky_operation()
except ValueError as e:
    # 捕获 ValueError
    print(f"值错误: {e}")
except KeyError as e:
    # 捕获 KeyError
    print(f"键错误: {e}")
except Exception as e:
    # 捕获所有其他异常
    print(f"未知错误: {e}")
else:
    # 没有异常时执行
    print(f"操作成功: {result}")
finally:
    # 无论是否异常都执行（释放资源）
    print("清理资源")
```

5.2.3 常见异常类型

异常类型	说明
ValueError	值错误（如 int('abc')）
TypeError	类型错误（如 1 + 'a'）
KeyError	键错误（字典中不存在的键）
IndexError	索引错误（列表越界）
ZeroDivisionError	除零错误
FileNotFoundError	文件不存在
PermissionError	权限错误
IOError	IO错误
AttributeError	属性错误（对象没有该属性）
ImportError	导入错误

5.2.4 自定义异常

```
class DataCleanError(Exception):
    """数据清洗异常。"""
    def __init__(self, message, step=None):
        super().__init__(message)
        self.step = step  # 出错的步骤

# 使用
try:
    raise DataCleanError("基站关联失败", step="关联")
except DataCleanError as e:
    print(f"数据清洗错误（步骤{e.step}）：{e}")
```

5.2.5 异常处理最佳实践

1. **捕获具体异常**：不要裸 `except:`，捕获具体异常类型；
2. **不要吞异常**：捕获后至少记录日志，不要静默忽略；
3. **尽量缩小 try 范围**：只包裹可能出错的代码；

4. **finally 释放资源**：文件、连接等在 finally 中释放；
5. **自定义异常**：业务异常用自定义异常，更清晰。

5.3 完整代码（异常处理实战）

```
# ============================================================
# 课时5：异常处理实战
# ============================================================

import os
import logging
import pandas as pd
import numpy as np

# 配置日志
logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)


# ============================================================
# 1. 自定义异常
# ============================================================
class DataProcessError(Exception):
    """数据处理异常。"""
    def __init__(self, message, module=None, step=None):
        super().__init__(message)
        self.module = module  # 出错的模块
        self.step = step      # 出错的步骤


class ConfigError(Exception):
    """配置异常。"""
    pass


# ============================================================
# 2. 安全读取文件
# ============================================================
def safe_read_csv(file_path, **kwargs):
    """
```

```
    安全读取CSV文件，处理各种异常。

    参数:
        file_path: 文件路径
        **kwargs: 其他参数

    返回:
        DataFrame（失败返回None）
    """
    try:
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"文件不存在: {file_path}")

        if not os.path.isfile(file_path):
            raise ValueError(f"不是文件: {file_path}")

        df = pd.read_csv(file_path, **kwargs)
        logger.info(f"读取成功: {file_path} ({len(df)}行)")
        return df

    except FileNotFoundError as e:
        logger.error(f"文件不存在: {e}")
        return None
    except PermissionError as e:
        logger.error(f"权限不足: {e}")
        return None
    except pd.errors.EmptyDataError:
        logger.error(f"文件为空: {file_path}")
        return None
    except pd.errors.ParserError as e:
        logger.error(f"解析错误: {e}")
        return None
    except Exception as e:
        logger.error(f"未知错误: {type(e).__name__}: {e}")
        return None


# ============================================================
```

```
# 3. 安全数据清洗
# ============================================================
def safe_data_clean(df):
    """
    安全数据清洗，处理各种异常。

    参数:
        df: 原始DataFrame

    返回:
        清洗后的DataFrame（失败返回None）
    """
    try:
        if df is None:
            raise DataProcessError("输入数据为空", module="data_clean", step="检查")

        if len(df) == 0:
            raise DataProcessError("输入数据为空", module="data_clean", step="检查")

        # 检查必要列
        required_cols = ["imsi", "time", "lac_id", "cell_id"]
        missing_cols = [c for c in required_cols if c not in df.columns]
        if missing_cols:
            raise DataProcessError(f"缺少必要列: {missing_cols}",
                                   module="data_clean", step="检查列")

        # 去空值
        df = df.dropna(subset=["lac_id", "cell_id"])
        df = df[(df["lac_id"] != "") & (df["cell_id"] != "")]

        if len(df) == 0:
            logger.warning("清洗后数据为空")
            return df

        logger.info(f"数据清洗完成: {len(df)}行")
        return df

    except DataProcessError as e:
```

```
        logger.error(f"数据处理错误（{e.module}/{e.step}）: {e}")
        return None
    except Exception as e:
        logger.error(f"未知错误: {type(e).__name__}: {e}")
        return None


# ============================================================
# 4. 安全数值转换
# ============================================================
def safe_to_numeric(series, default=np.nan):
    """
    安全转换为数值类型。

    参数:
        series: pandas Series
        default: 转换失败时的默认值

    返回:
        转换后的Series
    """
    try:
        result = pd.to_numeric(series, errors="coerce")
        failed_count = result.isna().sum() - series.isna().sum()
        if failed_count > 0:
            logger.warning(f"数值转换失败: {failed_count}个值")
        return result.fillna(default)
    except Exception as e:
        logger.error(f"数值转换错误: {e}")
        return pd.Series([default] * len(series))


# ============================================================
# 5. 上下文管理器（with语句）
# ============================================================
class SafeFileHandler:
    """安全文件处理器（上下文管理器）。"""
```

```
    def __init__(self, file_path, mode="r"):
        self.file_path = file_path
        self.mode = mode
        self.file = None

    def __enter__(self):
        """进入with语句时打开文件。"""
        try:
            self.file = open(self.file_path, self.mode, encoding="utf-8")
            logger.info(f"打开文件: {self.file_path}")
            return self.file
        except Exception as e:
            logger.error(f"打开文件失败: {e}")
            raise

    def __exit__(self, exc_type, exc_val, exc_tb):
        """退出with语句时关闭文件。"""
        if self.file:
            self.file.close()
            logger.info(f"关闭文件: {self.file_path}")
        # 返回False表示不抑制异常
        return False


# ============================================================
# 6. 演示
# ============================================================
def demo_exception_handling():
    """演示异常处理。"""
    print("=" * 60)
    print("异常处理演示")
    print("=" * 60)

    # 1. 安全读取文件
    print("\n1. 安全读取文件:")
    df = safe_read_csv("不存在的文件.csv")
    print(f"  读取不存在的文件: {df}")
```

```
# 2. 安全数据清洗
print("\n2. 安全数据清洗:")
df_empty = pd.DataFrame()
result = safe_data_clean(df_empty)
print(f"  清洗空DataFrame: {result}")

df_missing = pd.DataFrame({"a": [1, 2]})
result = safe_data_clean(df_missing)
print(f"  清洗缺少列的DataFrame: {result}")

# 3. 安全数值转换
print("\n3. 安全数值转换:")
s = pd.Series(["1", "2", "abc", "4"])
result = safe_to_numeric(s)
print(f"  原始: {s.tolist()}")
print(f"  转换后: {result.tolist()}")

# 4. 自定义异常
print("\n4. 自定义异常:")
try:
    raise DataProcessError("测试异常", module="test", step="demo")
except DataProcessError as e:
    print(f"  捕获自定义异常: {e} (模块={e.module}, 步骤={e.step})")

# 5. try-except-else-finally
print("\n5. try-except-else-finally:")
for x in [1, 0, "a"]:
    try:
        result = 10 / x
    except ZeroDivisionError:
        print(f"  10 / {x}: 除零错误")
    except TypeError as e:
        print(f"  10 / {x}: 类型错误 - {e}")
    else:
        print(f"  10 / {x} = {result} (成功)")
    finally:
        print(f"    (finally 总是执行)")
```

```
if __name__ == "__main__":
    demo_exception_handling()
```

5.4 运行结果与解读

运行后展示异常处理的各种用法：
1. 安全读取文件：处理文件不存在、权限不足、空文件、解析错误；
2. 安全数据清洗：检查输入、检查列、处理空数据；
3. 安全数值转换：处理转换失败的值；
4. 自定义异常：DataProcessError 携带模块和步骤信息；
5. try-except-else-finally：完整结构演示。

> 关键理解：
> 1. 异常处理让程序更健壮，遇到错误不会崩溃；
> 2. 捕获具体异常，不要裸 except；
> 3. 不要吞异常，至少记录日志；
> 4. finally 确保资源释放；
> 5. 自定义异常让业务错误更清晰。

5.5 当堂练习

1. 为 tr_stage2.py 增加异常处理（处理K值过大、空数据等）；
2. 自定义一个 ClusterError 异常，在聚类失败时抛出；
3. 思考：什么情况下应该捕获异常，什么情况下应该让异常抛出？

5.6 课时小结

- 异常处理：try-except-else-finally；
- 常见异常：ValueError、TypeError、KeyError、IndexError、FileNotFoundError；
- 自定义异常：继承 Exception，携带额外信息；
- 最佳实践：捕获具体异常、不吞异常、缩小try范围、finally释放资源；
- 上下文管理器：with 语句确保资源释放。

课时6：代码规范与静态检查

6.1 课时目标

掌握代码规范和静态检查工具，提高代码质量。

6.2 知识点深度讲解

6.2.1 为什么需要代码规范

代码规范的意义：
1. **可读性**：统一的风格让代码更容易阅读；
2. **可维护性**：规范的代码更容易修改和扩展；
3. **协作开发**：团队成员遵循相同规范，减少沟通成本；
4. **减少bug**：规范的代码更少出错；
5. **专业形象**：规范的代码体现专业素养。

6.2.2 PEP8 规范

PEP8 是 Python 官方代码风格指南：
1. **缩进**：4个空格，不用Tab；
2. **行长度**：最大79字符（文档字符串/注释72）；
3. **空行**：函数之间2个空行，方法之间1个空行；
4. **导入**：标准库→第三方→本地，每组之间空行；
5. **命名**：
- 函数/变量：snake_case（小写+下划线）；
- 类：PascalCase（大驼峰）；
- 常量：UPPER_CASE（大写+下划线）；
- 私有：_leading_underscore（单下划线开头）；
6. **空格**：运算符两侧空格，逗号后空格；
7. **注释**：# 后空格，文档字符串用三引号。

6.2.3 静态检查工具

1. **flake8**：检查PEP8规范、语法错误、未使用的导入；
2. **pylint**：更全面的检查，包括代码复杂度、命名规范、错误检测；
3. **black**：自动格式化代码，不用手动调整；
4. **isort**：自动排序导入；
5. **mypy**：类型检查（需要类型注解）。

6.2.4 文档字符串规范

```
def function_name(param1, param2):
    """
    函数功能简述。

    详细说明（可选）。

    参数:
        param1 (int): 参数1说明
        param2 (str): 参数2说明

    返回:
        bool: 返回值说明

    异常:
        ValueError: 参数无效时抛出

    示例:
        >>> function_name(1, "test")
        True
    """
    pass
```

6.3 完整代码（代码规范示例）

```
# ============================================================
# 课时6：代码规范示例
# 符合 PEP8 规范的代码
# ============================================================

"""
交通时空大数据分析系统 - 工具模块

本模块提供通用工具函数，包括距离计算、时间解析、数据读写等。

作者：魏波微
日期：2026-10-03
"""

# ============================================================
# 标准库导入
# ============================================================
import os
import time
import logging
from datetime import datetime

# ============================================================
# 第三方库导入
# ============================================================
import numpy as np
import pandas as pd

# ============================================================
# 本地模块导入
# ============================================================
# from config import DATA_DIR  # 示例


# ============================================================
# 常量定义（大写+下划线）
```

```
# ============================================================
EARTH_RADIUS_KM = 6371.0  # 地球半径（千米）
DEFAULT_ENCODING = "utf-8"  # 默认编码



# ============================================================
# 日志配置
# ============================================================
logger = logging.getLogger(__name__)



# ============================================================
# 类定义（大驼峰命名）
# ============================================================
class Timer:
    """
    计时器上下文管理器，用于统计代码执行时间。

    属性:
        name (str): 计时器名称
        start_time (float): 开始时间

    示例:
        >>> with Timer("测试"):
        ...     time.sleep(0.1)
    """

    def __init__(self, name="任务"):
        """
        初始化计时器。

        参数:
            name (str): 计时器名称，默认"任务"
        """
        self.name = name
        self.start_time = None

    def __enter__(self):
```

```
        """进入with语句，记录开始时间。"""
        self.start_time = time.perf_counter()
        logger.info(f"{self.name} 开始")
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        """退出with语句，计算耗时。"""
        elapsed = time.perf_counter() - self.start_time
        logger.info(f"{self.name} 完成，耗时: {elapsed:.2f}秒")
        return False


# ============================================================
# 函数定义（snake_case命名）
# ============================================================
def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """
    计算球面上两点之间的距离（Haversine公式）。

    参数:
        lon1 (float): 第一点经度
        lat1 (float): 第一点纬度
        lon2 (float or np.ndarray): 第二点经度（支持数组）
        lat2 (float or np.ndarray): 第二点纬度（支持数组）
        unit (str): 距离单位，"km"或"m"，默认"km"

    返回:
        float or np.ndarray: 距离（与输入形状一致）

    异常:
        ValueError: unit参数无效时抛出

    示例:
        >>> haversine(123.43, 41.80, 116.40, 39.90)
        680.5
    """
    # 参数校验
    if unit not in ("km", "m"):
```

```
        raise ValueError(f"无效的单位: {unit}，应为 'km' 或 'm'")

    # 角度转弧度
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])

    # 差值
    dlat = lat2 - lat1
    dlon = lon2 - lon1

    # Haversine公式
    a = (np.sin(dlat / 2) ** 2 +
         np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2)
    c = 2 * np.arcsin(np.sqrt(a))

    # 距离
    distance = EARTH_RADIUS_KM * c

    # 单位转换
    return distance * 1000 if unit == "m" else distance


def parse_time(time_str, time_format="%Y%m%d%H%M%S"):
    """
    解析时间字符串为 datetime 对象。

    参数:
        time_str (str): 时间字符串
        time_format (str): 时间格式，默认"%Y%m%d%H%M%S"

    返回:
        datetime or pd.NaT: 解析后的datetime（失败返回NaT）

    示例:
        >>> parse_time("20181003120000")
        datetime.datetime(2018, 10, 3, 12, 0)
    """
    return pd.to_datetime(time_str, format=time_format, errors="coerce")
```

```
def save_csv(df, file_path, index=False, encoding=DEFAULT_ENCODING):
    """
    保存DataFrame为CSV文件。

    参数:
        df (pd.DataFrame): 要保存的DataFrame
        file_path (str): 文件路径
        index (bool): 是否保存索引，默认False
        encoding (str): 文件编码，默认"utf-8"

    返回:
        None
    """
    # 确保目录存在
    os.makedirs(os.path.dirname(file_path), exist_ok=True)
    df.to_csv(file_path, index=index, encoding=encoding)
    logger.info(f"保存: {file_path} ({len(df)}行)")


# ============================================================
# 私有函数（单下划线开头）
# ============================================================
def _validate_coordinates(lon, lat):
    """
    校验经纬度是否有效。

    参数:
        lon (float): 经度
        lat (float): 纬度

    返回:
        bool: 是否有效
    """
    return -180 <= lon <= 180 and -90 <= lat <= 90


# ============================================================
```

```python
# 主函数
# ============================================================
def main():
    """主函数：模块自测。"""
    print("=" * 60)
    print("工具模块自测")
    print("=" * 60)

    # 测试 Haversine
    distance = haversine(123.43, 41.80, 116.40, 39.90)
    print(f"沈阳到北京: {distance:.1f} km")

    # 测试 Timer
    with Timer("测试任务"):
        time.sleep(0.1)

    print("自测完成！")


if __name__ == "__main__":
    main()
```

6.4 静态检查工具使用

```
# 安装工具
pip install flake8 pylint black isort

# flake8 检查
flake8 utils.py

# pylint 检查（更详细）
pylint utils.py

# black 自动格式化
black utils.py

# isort 排序导入
isort utils.py

# 一次性检查整个项目
flake8 project_code/
pylint project_code/
```

6.5 运行结果与解读

- 代码符合 PEP8 规范：4空格缩进、79字符行宽、snake_case函数名、PascalCase类名；
- 文档字符串完整：函数功能、参数、返回值、异常、示例；
- 导入分组：标准库→第三方→本地；
- 私有函数用单下划线开头；
- 常量用大写+下划线。

> 关键理解：
> 1. PEP8 是 Python 官方规范，应该遵循；
> 2. 文档字符串是代码的重要组成部分，不能省略；
> 3. 静态检查工具能自动发现问题，推荐使用；
> 4. black 自动格式化，不用手动调整风格；
> 5. 规范的代码更专业、更易维护。

6.6 当堂练习

1. 用 flake8 检查 project_code 下的所有文件，修复警告；
2. 用 black 格式化所有 Python 文件；
3. 为 data_clean.py 补充完整的文档字符串。

6.7 课时小结

- PEP8 规范：4空格缩进、79字符行宽、命名规范、导入分组；
- 文档字符串：功能、参数、返回值、异常、示例；
- 静态检查：flake8（规范）、pylint（全面）、black（格式化）、isort（导入排序）；
- 命名规范：函数snake_case、类PascalCase、常量UPPER_CASE、私有_leading；
- 规范的代码更专业、更易维护。

课时7：版本控制（git）

7.1 课时目标

掌握 git 版本控制，管理代码历史和协作。

7.2 知识点深度讲解

7.2.1 为什么需要版本控制

版本控制的意义：
1. **历史记录**：记录每次修改，随时回退；
2. **协作开发**：多人同时开发，合并代码；
3. **分支管理**：新功能在分支开发，不影响主线；
4. **版本发布**：标记版本，方便发布和回滚；
5. **代码审查**：提交前审查代码，保证质量。

7.2.2 git 基本概念

1. **工作区（Working Directory）**：当前编辑的文件；
2. **暂存区（Staging Area）**：git add 后的文件；
3. **本地仓库（Local Repository）**：git commit 后的历史；
4. **远程仓库（Remote Repository）**：GitHub/GitLab 上的仓库；
5. **分支（Branch）**：独立的开发线；
6. **提交（Commit）**：一次修改的快照；

7. **HEAD**：当前分支的最新提交。

7.2.3 常用 git 命令

```
# 初始化
git init                    # 初始化仓库
git clone <url>             # 克隆远程仓库

# 基本操作
git status                  # 查看状态
git add <file>              # 添加到暂存区
git add .                   # 添加所有文件
git commit -m "message"     # 提交
git diff                    # 查看修改

# 历史查看
git log                     # 查看提交历史
git log --oneline           # 简洁历史
git show <commit>           # 查看某次提交

# 分支操作
git branch                  # 查看分支
git branch <name>           # 创建分支
git checkout <name>         # 切换分支
git switch <name>           # 切换分支（新命令）
git merge <branch>          # 合并分支
git branch -d <name>        # 删除分支

# 远程操作
git remote -v               # 查看远程仓库
git push origin <branch>    # 推送到远程
git pull origin <branch>    # 从远程拉取
git fetch origin            # 获取远程更新

# 撤销操作
git checkout -- <file>      # 撤销工作区修改
git reset HEAD <file>       # 撤销暂存
git reset --hard <commit>   # 回退到某次提交（危险）
git revert <commit>         # 撤销某次提交（生成新提交）
```

7.2.4 提交信息规范

好的提交信息：

```
<type>(<scope>): <subject>

<body>

<footer>
```

type 类型：
- feat：新功能
- fix：修复bug
- docs：文档
- style：格式（不影响代码运行）
- refactor：重构
- test：测试
- chore：构建/工具

示例：

```
feat(出行驻留): 添加k-means聚类最优K选择

- 支持SSE、轮廓系数、邓恩指数三种评估指标
- 遍历K=2~10自动选择最优K
- 添加单点成簇判定逻辑

Closes #12
```

7.2.5 .gitignore

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
*.egg-info/
dist/
build/

# 虚拟环境
venv/
env/
.venv/

# IDE
.idea/
.vscode/
*.swp
*.swo

# 数据文件（大文件不提交）
*.csv
*.xlsx
data/
result/

# 日志
*.log
logs/

# 系统文件
.DS_Store
Thumbs.db
```

7.3 完整代码（git 工作流演示）

```
# ============================================================
# 课时7：git 工作流演示
# ============================================================

# 1. 初始化项目
mkdir traffic-analysis
cd traffic-analysis
git init
git config user.name "魏波微"
git config user.email "weibowei@example.com"

# 2. 创建 .gitignore
cat > .gitignore << 'EOF'
__pycache__/
*.pyc
venv/
.idea/
.vscode/
result/
*.csv
*.log
.DS_Store
EOF

# 3. 第一次提交
git add .
git commit -m "chore: 初始化项目，添加.gitignore"

# 4. 开发新功能（创建分支）
git checkout -b feature/data-clean

# 5. 编写代码
# ... 编写 data_clean.py ...

# 6. 提交
git add data_clean.py
```

```
git commit -m "feat(数据清洗): 实现9条清洗规则

- 字段抽取、去脏imsi、去空值
- 时间转换、日期过滤、基站关联
- 排序去重、单用户过滤"

# 7. 继续开发
# ... 修改代码 ...
git add data_clean.py
git commit -m "fix(数据清洗): 修复lac_id类型推断问题

- lac_id/cell_id必须用dtype=str读取
- 否则pandas推断为float，与基站表拼接对不上"

# 8. 合并到主分支
git checkout main
git merge feature/data-clean
git branch -d feature/data-clean

# 9. 查看历史
git log --oneline --graph

# 10. 推送到远程（如果有远程仓库）
# git remote add origin https://github.com/user/traffic-analysis.git
# git push -u origin main
```

7.4 运行结果与解读

git 工作流演示:
1. 初始化项目，配置用户信息;
2. 创建 .gitignore，忽略不需要提交的文件;
3. 第一次提交;
4. 创建功能分支，开发新功能;
5. 提交代码，提交信息遵循规范;
6. 修复bug，提交修复;
7. 合并到主分支，删除功能分支;
8. 查看提交历史。

> 关键理解：
> 1. git 是分布式版本控制系统，每个开发者都有完整历史；
> 2. 工作区→暂存区→本地仓库→远程仓库；
> 3. 分支是 git 的核心，新功能在分支开发；
> 4. 提交信息要规范，type(scope): subject；
> 5. .gitignore 忽略不需要提交的文件（数据、缓存、IDE配置）。

7.5 当堂练习

1. 在项目目录初始化 git 仓库；
2. 创建 .gitignore 文件；
3. 提交所有代码（不包括数据文件和结果文件）；
4. 创建一个 feature 分支，修改代码后提交，再合并回 main。

7.6 课时小结

- git 基本概念：工作区、暂存区、本地仓库、远程仓库、分支、提交；
- 常用命令：init/clone/add/commit/status/log/branch/merge/push/pull；
- 分支工作流：创建分支→开发→提交→合并→删除；
- 提交信息规范：type(scope): subject，feat/fix/docs/style/refactor/test/chore；
- .gitignore：忽略数据、缓存、IDE配置等不需要提交的文件。

课时8：系统测试实战

8.1 课时目标

综合运用测试、调试、异常处理、代码规范，完成系统测试实战。

8.2 知识点深度讲解

8.2.1 系统测试流程

系统测试流程：
1. **制定测试计划**：确定测试范围、策略、资源；
2. **编写测试用例**：单元测试、集成测试；
3. **执行测试**：运行测试用例，记录结果；
4. **缺陷管理**：记录、跟踪、修复缺陷；

5. **性能测试**：测量系统性能；
6. **代码审查**：检查代码规范和质量；
7. **测试报告**：总结测试结果和质量评估。

8.2.2 本系统的测试清单

1. **单元测试**：
   - utils.py：haversine、parse_time、save_csv；
   - data_clean.py：各清洗规则；
   - tr_stage1/2/3.py：各阶段算法；
   - purpose.py、mode.py：识别规则；
2. **集成测试**：
   - 数据清洗→阶段1→阶段2→阶段3→目的/方式；
   - main.py 全流程；
3. **性能测试**：
   - 各模块执行时间；
   - 全流程总时间；
   - 内存占用；
4. **代码规范**：
   - flake8 检查；
   - 文档字符串完整性；
5. **异常处理**：
   - 空数据、缺失列、文件不存在等边界情况。

8.3 完整代码（系统测试实战）

```
# ============================================================
# 课时8：系统测试实战
# 文件：run_all_tests.py
# 运行：python run_all_tests.py
# ============================================================

import os
import sys
import time
import subprocess
import pandas as pd
import numpy as np

# 添加代码目录
CODE_DIR = os.path.join(os.path.dirname(__file__), "project_code")
sys.path.insert(0, CODE_DIR)

from config import (
    CLEANED_DATA_FILE, STAGE1_TRAVEL_POINTS_FILE,
    STAGE2_CLUSTER_FILE, STAGE3_LABEL_FILE,
    PURPOSE_RESULT_FILE, MODE_RESULT_FILE, HEATMAP_RESULT_FILE
)



class TestResult:
    """测试结果收集器。"""

    def __init__(self):
        self.passed = []
        self.failed = []
        self.errors = []

    def add_pass(self, name):
        self.passed.append(name)
        print(f"  ✓ {name}")
```

```
    def add_fail(self, name, reason):
        self.failed.append((name, reason))
        print(f"  ✗ {name}: {reason}")

    def add_error(self, name, error):
        self.errors.append((name, error))
        print(f"  ✗ {name}: {type(error).__name__}: {error}")

    def summary(self):
        total = len(self.passed) + len(self.failed) + len(self.errors)
        print(f"\n{'='*60}")
        print(f"测试总结: 通过{len(self.passed)}, 失败{len(self.failed)}, 错误
{len(self.errors)}, 共{total}")
        print(f"{'='*60}")
        return len(self.failed) == 0 and len(self.errors) == 0


def test_file_exists(result):
    """测试1: 结果文件存在。"""
    print("\n[测试1] 结果文件存在")
    files = {
        "cleaned_data.csv": CLEANED_DATA_FILE,
        "stage1_travel_points.csv": STAGE1_TRAVEL_POINTS_FILE,
        "stage2_cluster.csv": STAGE2_CLUSTER_FILE,
        "stage3_label.csv": STAGE3_LABEL_FILE,
        "purpose_result.csv": PURPOSE_RESULT_FILE,
        "mode_result.csv": MODE_RESULT_FILE,
        "heatmap_result.csv": HEATMAP_RESULT_FILE,
    }
    for name, path in files.items():
        if os.path.exists(path):
            result.add_pass(f"文件存在: {name}")
        else:
            result.add_fail(f"文件存在: {name}", f"文件不存在: {path}")


def test_data_format(result):
    """测试2: 数据格式正确。"""
```

```
    print("\n[测试2] 数据格式正确")

    # cleaned_data.csv
    try:
        df = pd.read_csv(CLEANED_DATA_FILE, nrows=5)
        required = ["imsi", "time", "lac_id", "cell_id", "lon", "lat"]
        missing = [c for c in required if c not in df.columns]
        if missing:
            result.add_fail("cleaned_data列", f"缺少列: {missing}")
        else:
            result.add_pass("cleaned_data列完整")
    except Exception as e:
        result.add_error("cleaned_data读取", e)

    # stage3_label.csv
    try:
        df = pd.read_csv(STAGE3_LABEL_FILE, nrows=5)
        if "label" in df.columns:
            result.add_pass("stage3_label有label列")
        else:
            result.add_fail("stage3_label列", "缺少label列")
    except Exception as e:
        result.add_error("stage3_label读取", e)


def test_data_validity(result):
    """测试3：数据有效性。"""
    print("\n[测试3] 数据有效性")

    # stage3 标签值
    try:
        df = pd.read_csv(STAGE3_LABEL_FILE)
        labels = set(df["label"].unique())
        if labels.issubset({0, 1}):
            result.add_pass(f"stage3标签值有效: {labels}")
        else:
            result.add_fail("stage3标签值", f"非法标签: {labels}")
```

```
        # 两种标签都存在
        if 0 in labels and 1 in labels:
            result.add_pass("stage3两种标签都存在")
        else:
            result.add_fail("stage3标签", f"缺少标签: {labels}")
    except Exception as e:
        result.add_error("stage3有效性", e)

    # cleaned_data 无空坐标
    try:
        df = pd.read_csv(CLEANED_DATA_FILE)
        if df["lon"].notna().all() and df["lat"].notna().all():
            result.add_pass("cleaned_data无空坐标")
        else:
            result.add_fail("cleaned_data坐标", "存在空坐标")
    except Exception as e:
        result.add_error("cleaned_data有效性", e)


def test_performance(result):
    """测试4：性能测试。"""
    print("\n[测试4] 性能测试")

    # 读取 cleaned_data 性能
    try:
        start = time.perf_counter()
        df = pd.read_csv(CLEANED_DATA_FILE)
        elapsed = time.perf_counter() - start
        if elapsed < 10:
            result.add_pass(f"读取cleaned_data: {elapsed:.2f}秒 (<10秒)")
        else:
            result.add_fail("读取性能", f"耗时过长: {elapsed:.2f}秒")
    except Exception as e:
        result.add_error("读取性能", e)

    # 文件大小
    try:
        size = os.path.getsize(CLEANED_DATA_FILE) / 1024 / 1024
```

```
        result.add_pass(f"cleaned_data大小: {size:.2f} MB")
    except Exception as e:
        result.add_error("文件大小", e)


def test_code_quality(result):
    """测试5: 代码质量（flake8）。"""
    print("\n[测试5] 代码质量（flake8）")

    try:
        # 检查 flake8 是否安装
        r = subprocess.run(["python", "-m", "flake8", "--version"],
                           capture_output=True, text=True, timeout=10)
        if r.returncode != 0:
            result.add_fail("flake8", "未安装 flake8")
            return

        # 运行 flake8
        r = subprocess.run(["python", "-m", "flake8", CODE_DIR,
                            "--max-line-length=100", "--ignore=E501,W503"],
                           capture_output=True, text=True, timeout=30)

        if r.returncode == 0:
            result.add_pass("flake8检查通过")
        else:
            # 统计警告数量
            warnings = len(r.stdout.strip().split("\n")) if r.stdout.strip() else 0
            result.add_fail("flake8", f"{warnings}个警告")
            # 显示前5个
            if r.stdout:
                for line in r.stdout.strip().split("\n")[:5]:
                    print(f"    {line}")
    except subprocess.TimeoutExpired:
        result.add_fail("flake8", "超时")
    except Exception as e:
        result.add_error("flake8", e)
```

```
def run_all_tests():
    """运行所有测试。"""
    print("=" * 70)
    print("  系统测试实战")
    print("=" * 70)

    result = TestResult()

    test_file_exists(result)
    test_data_format(result)
    test_data_validity(result)
    test_performance(result)
    test_code_quality(result)

    all_passed = result.summary()

    if all_passed:
        print("\n🎉 所有测试通过！系统质量良好。")
    else:
        print("\n⚠️  存在测试失败，请修复后重新运行。")

    return all_passed


if __name__ == "__main__":
    success = run_all_tests()
    sys.exit(0 if success else 1)
```

8.4 运行结果与解读

运行后展示系统测试的完整结果：
1. 结果文件存在：7个结果文件检查；
2. 数据格式正确：必要列检查；
3. 数据有效性：标签值、无空坐标；
4. 性能测试：读取时间、文件大小；
5. 代码质量：flake8 检查；
6. 测试总结：通过/失败/错误统计。

> 关键理解：
> 1. 系统测试是全方位的质量保证；
> 2. 测试清单覆盖文件存在、格式、有效性、性能、代码质量；
> 3. 自动化测试可以反复运行，确保每次修改不引入新问题；
> 4. 测试结果是系统质量的客观评估；
> 5. 测试失败时要及时修复，不要带病上线。

8.5 当堂练习

1. 运行 run_all_tests.py，查看系统测试结果；
2. 修复测试中发现的问题；
3. 增加更多测试用例（如 purpose/mode 结果有效性）。

8.6 课时小结

- 系统测试流程：计划→用例→执行→缺陷→性能→审查→报告；
- 测试清单：单元测试、集成测试、性能测试、代码规范、异常处理；
- 自动化测试：TestResult 收集器，统一管理测试结果；
- 测试覆盖：文件存在、格式、有效性、性能、代码质量；
- 测试是质量保证，不是可选的。

三、本日小结

1. **软件测试基础**：意义、分类、流程、测试用例设计；
2. **单元测试**：unittest（标准库）、pytest（第三方，更简洁）；
3. **集成测试**：验证模块交互和数据流转；
4. **调试技巧**：print、logging、pdb 调试器；
5. **异常处理**：try-except-else-finally、自定义异常、最佳实践；
6. **代码规范**：PEP8、文档字符串、flake8/pylint/black；
7. **版本控制**：git 基本命令、分支工作流、提交规范、.gitignore；
8. **系统测试实战**：全方位测试，自动化测试框架。

四、本日作业

1. 为 utils.py 编写完整的单元测试（pytest）；

2. 为 data_clean.py 编写集成测试；
3. 用 flake8 检查所有代码，修复警告；
4. 用 black 格式化所有 Python 文件；
5. 在项目目录初始化 git，提交所有代码；
6. 运行 run_all_tests.py，确保所有测试通过；
7. 预习 Day14：项目答辩准备。